

Luồng gói tin trong hệ ArduPilot–Gazebo–ROS 2–RViz

Nguyễn Thanh Lâm

30 tháng 8 năm 2026

Tóm tắt

Báo cáo giải thích ở mức hệ thống cách dữ liệu cảm biến mô phỏng đi từ Gazebo đến RViz, đồng thời chỉ ra vòng điều khiển hai chiều giữa ArduPilot SITL và Gazebo. Trọng tâm là hình dạng gói tin, chuẩn mã hóa và phép biến đổi tại mỗi đầu nhận. Gazebo dùng Protocol Buffers trên Gazebo Transport; bridge chuyển đổi message có kiểu; ROS 2 dùng CDR và DDS; RViz là subscriber cuối. Trong vòng điều khiển, SITL gửi lệnh servo nhị phân sang plugin và nhận trạng thái mô phỏng ở dạng JSON. Các ví dụ raw được lấy từ lần chạy thực nghiệm, không phải dữ liệu minh họa tự tạo.

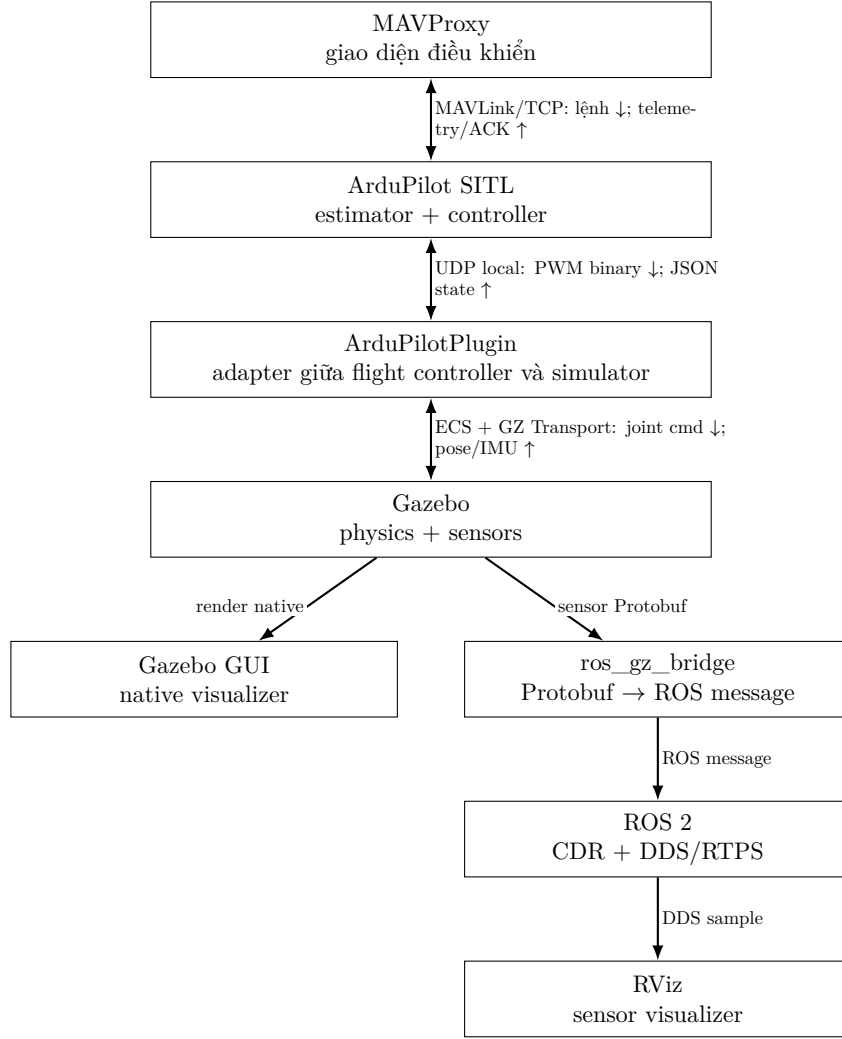
1 Câu hỏi cần trả lời

Hệ thống được khảo sát bắt đầu từ lệnh điều khiển của người dùng và kết thúc tại dữ liệu cảm biến hiển thị trong RViz. MAVProxy gửi lệnh GUIDED cho ArduPilot; ArduPilot điều khiển Gazebo qua plugin; Gazebo mô phỏng chuyển động và tạo phép đo; ROS 2 chuyển các phép đo đó đến RViz. Mỗi đoạn dùng một loại message và cơ chế truyền khác nhau. Báo cáo cần trả lời bốn câu hỏi:

1. Lệnh MAVProxy được mã hóa thành gói MAVLink nào và ArduPilot xử lý ra sao?
2. ArduPilot và ArduPilotPlugin trao đổi PWM, IMU và trạng thái mô phỏng bằng định dạng nào?
3. Plugin tác động lên Gazebo physics và nhận dữ liệu sensor qua ECS hoặc Gazebo Transport như thế nào?
4. Dữ liệu Gazebo được chuyển thành message ROS 2 và đi đến Gazebo GUI hoặc RViz ra sao?

Phương pháp kiểm chứng là chạy đồng thời MAVProxy, ArduPilot SITL, ArduPilotPlugin, Gazebo, bridge ROS 2 và RViz; sau đó gửi lệnh GUIDED, arm, takeoff và setpoint vận tốc. Việc UAV di chuyển trong Gazebo đồng thời làm ảnh camera, ảnh độ sâu và đám mây điểm trong RViz thay đổi, qua đó xác nhận toàn bộ chuỗi điều khiển và quan sát đang hoạt động trong thời gian thực.

2 Bức tranh tổng thể



Hình 1: Kiến trúc hoàn chỉnh từ lệnh điều khiển đến trực quan hóa sensor.

Hình 1 ghép bốn kết nối thành một hệ thống thống nhất. Lệnh điều khiển không đi thẳng tới RViz. Nó làm thay đổi đầu ra motor, lực tác dụng và trạng thái vật lý của UAV; trạng thái mới làm các sensor trong Gazebo tạo phép đo mới. Dữ liệu đó mới được đưa tới Gazebo GUI hoặc qua ROS 2 để RViz hiển thị.

	Kết nối	Dữ liệu trao đổi
A	MAVProxy ↔ ArduPilot	MAVLink mang mode, arm, takeoff, setpoint GUIDED theo chiều lệnh; ACK và telemetry đi theo chiều ngược lại.
B	ArduPilot ↔ ArduPilotPlugin	SITL gửi UDP binary chứa PWM; plugin trả UDP JSON chứa IMU, pose, quaternion và velocity.
C	ArduPilotPlugin ↔ Gazebo	Plugin ghi motor/joint command vào ECS; Gazebo physics trả pose và velocity qua ECS, còn IMU và sensor callback đi qua Gazebo Transport.
D	Gazebo → visualizer	Gazebo GUI đọc dữ liệu native. Nhánh RViz đi qua Protobuf/Gazebo Transport, bridge, message ROS 2 và CDR/DDS.

Bảng 1: Vai trò của bốn kết nối trong toàn hệ thống.

Có thể hình dung mỗi phép đo đi qua hai “ngôn ngữ”. Ở nửa đầu, Gazebo dùng message của Gazebo. Ở nửa sau, ROS 2 dùng các message chuẩn như *Image*, *Imu*, *LaserScan* và *PointCloud2*. Bridge đóng vai trò phiên dịch giữa hai ngôn ngữ này.

Cần tách hai đường truyền thường bị nhầm với nhau. Đường *điều khiển* tạo vòng kín ArduPilot–Gazebo và có dữ liệu ứng dụng theo cả hai chiều. Đường *quan sát* đưa sensor từ Gazebo tới RViz; dữ liệu

sensor đi một chiều, còn chiều ngược chỉ gồm discovery, subscription và acknowledgement của middleware khi QoS yêu cầu.

3 Các protocol và cơ chế chính

3.1 Protocol Buffers ở phía Gazebo

Protocol Buffers, thường gọi là Protobuf, là chuẩn tuần tự hóa dữ liệu có cấu trúc. Mỗi loại message có một schema mô tả tên trường, kiểu dữ liệu và số định danh của trường. Khi truyền đi, tên trường không được lặp lại dưới dạng văn bản; chúng được thay bằng các tag số và giá trị nhị phân. Vì vậy payload nhỏ hơn JSON và máy tính có thể giải mã nhanh hơn.

Ví dụ, một message IMU chứa các nhóm thông tin như dấu thời gian, hệ tọa độ, quaternion định hướng, vận tốc góc và gia tốc tuyến tính. Camera chứa kích thước ảnh, định dạng pixel và mảng byte. LiDAR chứa góc quét, khoảng đo hoặc một mảng điểm 3D.

3.2 Gazebo Transport

Gazebo Transport là lớp publish-subscribe của Gazebo. Sensor publish message lên một topic; thành phần quan tâm subscribe topic đó. Lớp này dùng ZeroMQ và kết nối TCP để vận chuyển dữ liệu chính. UDP multicast chủ yếu hỗ trợ discovery, tức giúp các tiến trình tìm thấy publisher, subscriber và topic của nhau.

Điểm quan trọng là Protobuf trả lời câu hỏi “các trường được mã hóa thế nào”, còn Gazebo Transport trả lời “message được tìm thấy và vận chuyển thế nào”. Hai khái niệm này bổ sung cho nhau nhưng không phải một.

3.3 Bridge Gazebo-ROS 2

Bridge không chuyển nguyên xi một chuỗi byte từ đầu này sang đầu kia. Nó giải mã message Gazebo, tạo một message ROS 2 mới rồi ánh xạ các trường tương ứng. Chẳng hạn, áp suất của Gazebo được đưa vào trường áp suất của ROS 2; quaternion, gia tốc và vận tốc góc được đưa vào các trường tương ứng của message IMU.

Một số metadata có thể không được giữ lại nếu phía ROS 2 không có trường tương ứng. Ngược lại, bridge có thể điền giá trị mặc định cho trường ROS 2 mà message Gazebo không cung cấp. Vì vậy đây là chuyển đổi ngữ nghĩa có kiểu, không chỉ là đổi header mạng.

3.4 DDS và CDR ở phía ROS 2

Sau bridge, dữ liệu trở thành message ROS 2. Middleware ROS 2 sử dụng DDS để discovery, quản lý publisher-subscriber và phân phối dữ liệu. CDR là định dạng nhị phân dùng để tuần tự hóa các trường của message ROS 2 trước khi truyền.

DDS còn cung cấp Quality of Service (QoS), ví dụ lựa chọn ưu tiên độ tin cậy hay độ trễ thấp. Với sensor tốc độ cao, cấu hình thường chấp nhận bỏ một số frame cũ để luôn hiển thị dữ liệu mới nhất. Publisher và subscriber phải có QoS tương thích thì dữ liệu mới đi qua.

3.5 RViz

RViz là subscriber ROS 2. Nó không hiểu Protobuf của Gazebo và không kết nối trực tiếp với sensor plugin. RViz nhận các message ROS 2 đã được middleware giải mã, sau đó chọn cách hiển thị theo loại message:

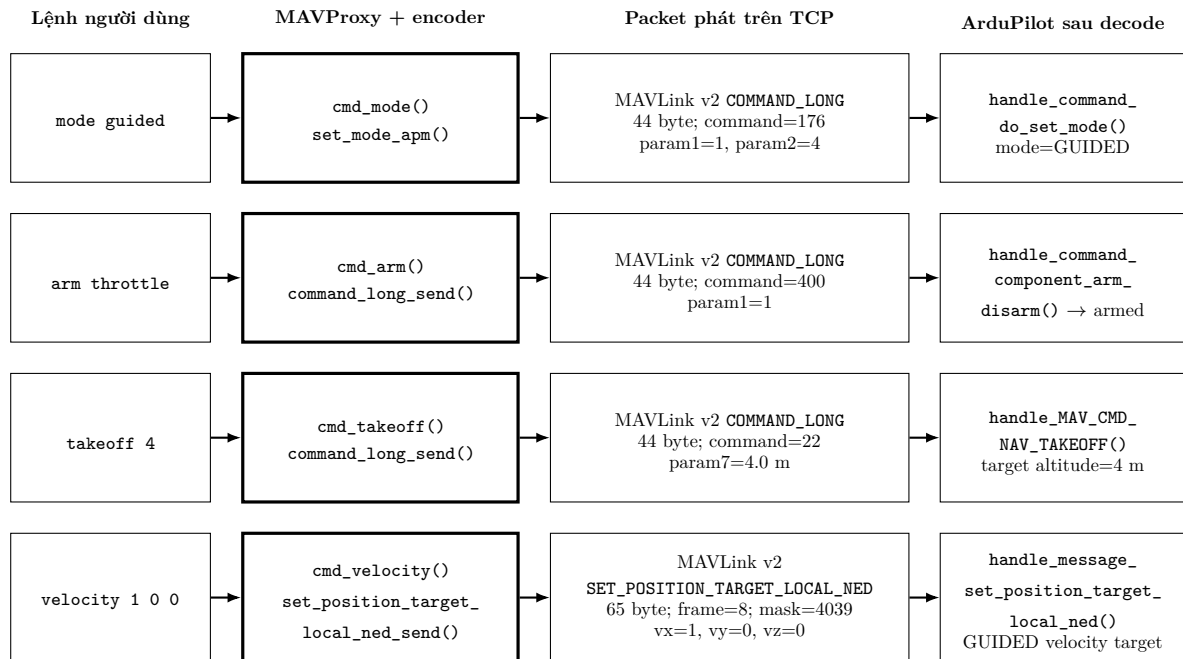
- **Image**: dựng ảnh RGB hoặc ảnh độ sâu;
- **LaserScan**: dựng lát cắt LiDAR 2D;
- **PointCloud2**: dựng đám mây điểm 3D;
- **Imu** và các số đo khác: hiển thị hướng, vector hoặc giá trị số.

Để đặt dữ liệu đúng vị trí trong không gian 3D, RViz còn cần `frame_id` và cây biến đổi tọa độ TF. Topic có dữ liệu nhưng thiếu TF phù hợp vẫn có thể khiến RViz không hiển thị gì.

4 Bốn kết nối với dữ liệu thật

Mỗi hình dưới đây chỉ dùng ba loại khối: dữ liệu trước xử lý, hàm xử lý và dữ liệu sau xử lý. Các giá trị packet lấy từ capture/rosbag của lần chạy thực nghiệm; riêng MAVLink được tái tạo bằng đúng encoder pymavlink mà MAVProxy trên máy đang sử dụng.

4.1 A. Kết nối MAVProxy và ArduPilot bằng MAVLink

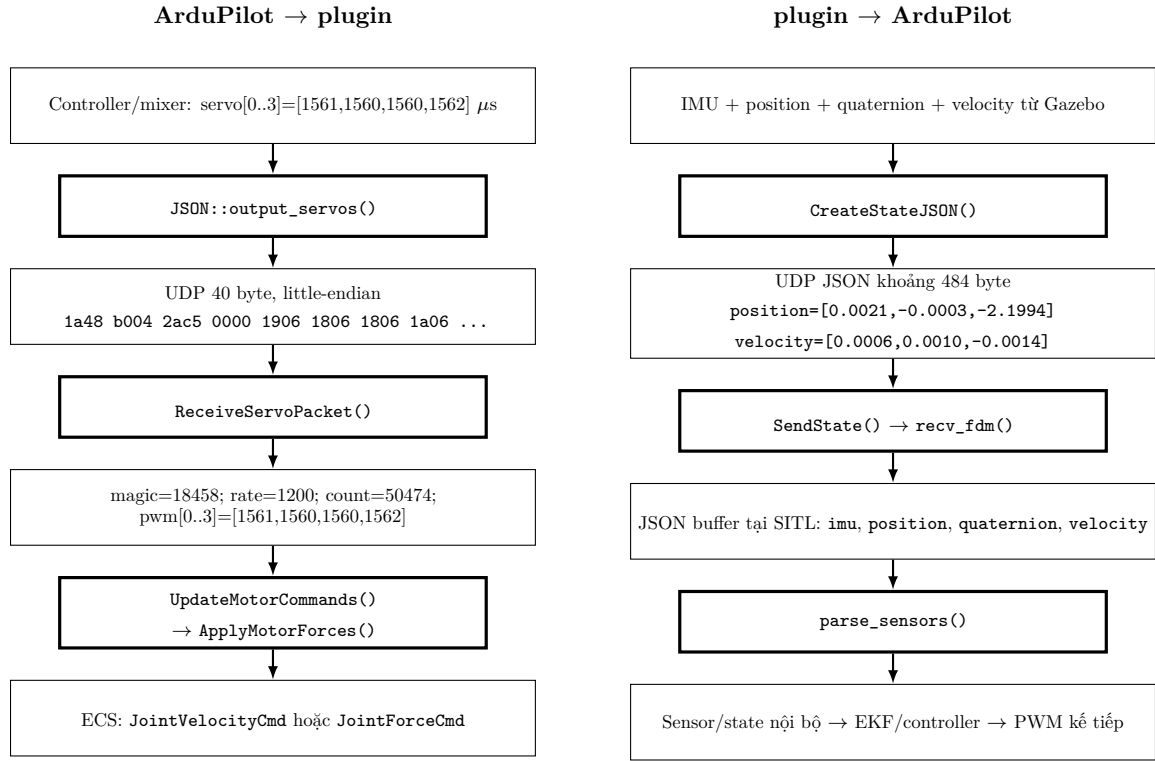


Hình 2: Bốn lệnh MAVProxy và packet MAVLink thật được tạo ra.

```
velocity 1 0 0 -> MAVLink v2 raw, seq=0:
fd 35 00 00 00 ff 00 54 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 80 3f 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 c7 0f 01
00 08 64 4f
```

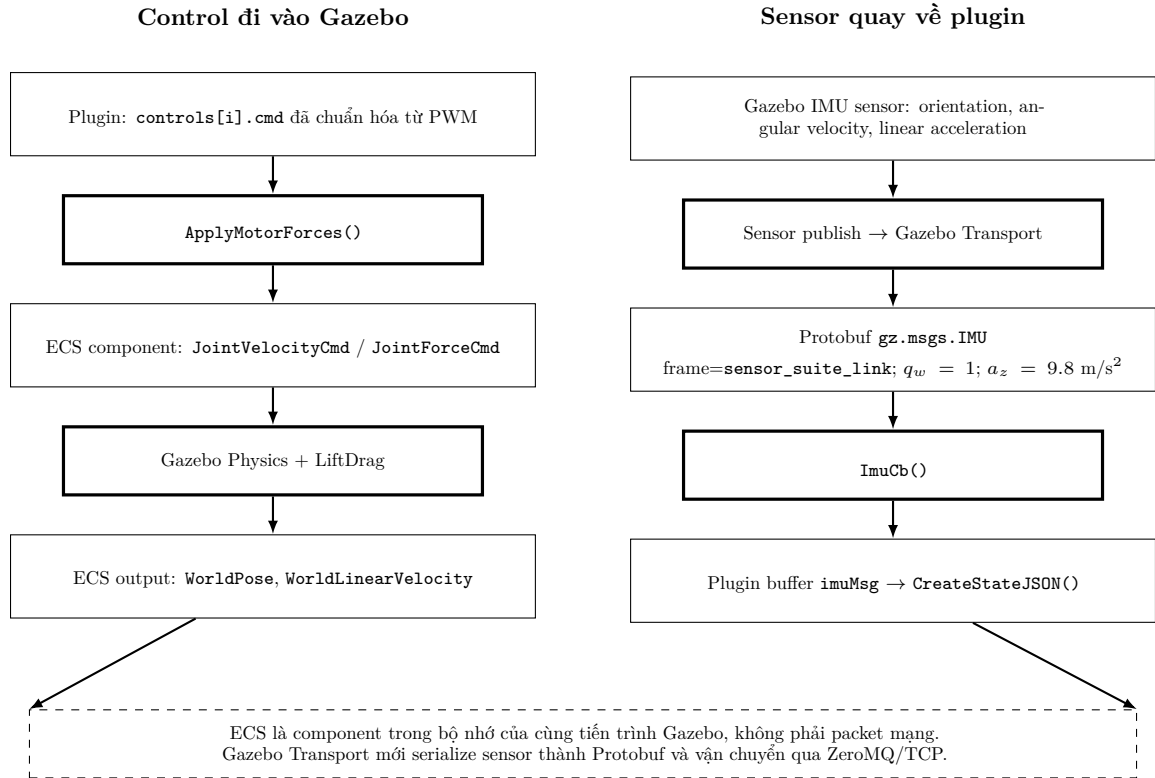
Byte đầu fd là MAVLink 2; 35 là payload 53 byte; message id 54 00 00=84; hai byte cuối là checksum. Sequence và checksum thay đổi theo phiên truyền, còn payload field giữ nguyên theo lệnh.

4.2 B. Kết nối ArduPilot và ArduPilotPlugin bằng UDP



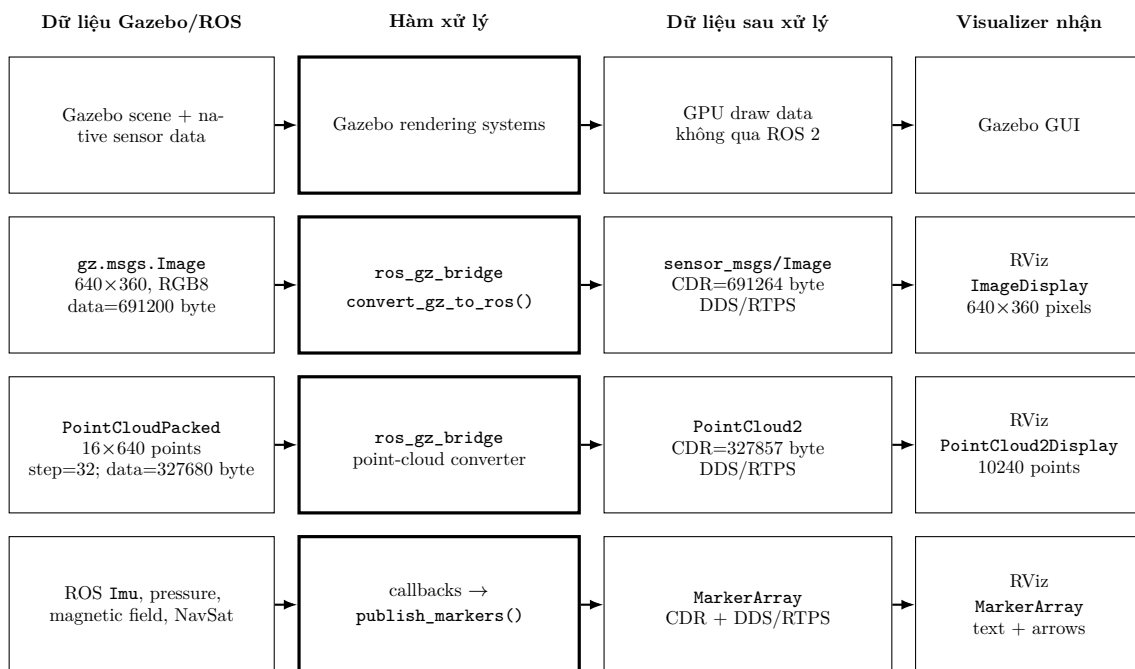
Hình 3: Hai chiều UDP với dữ liệu trước và sau từng hàm xử lý.

4.3 C. Kết nối ArduPilotPlugin và Gazebo bằng ECS và Gazebo Transport



Hình 4: Hai cơ chế khác nhau giữa plugin và Gazebo: ECS cho physics, Gazebo Transport cho sensor.

4.4 D. Gazebo và ROS 2 đưa dữ liệu đến visualizer



Hình 5: Bốn dữ liệu thật đi đến Gazebo GUI hoặc RViz.

Video và mã nguồn thực nghiệm

Video ghi trực tiếp Gazebo, RViz và UAV di chuyển bằng lệnh GUIDED: **xem video trên Google Drive**.

Mã nguồn, world cảm biến và cấu hình RViz: **GitHub repository**. Toàn bộ lệnh chạy lại nằm trong **runbook thực nghiệm**.

5 Cách xác định lỗi theo từng lớp

Khi RViz không hiển thị, nên kiểm tra theo thứ tự từ nguồn đến đích:

1. **Gazebo**: thế giới có đang chạy và sensor có tạo message không?
2. **Bridge**: topic Gazebo có được ánh xạ đúng sang loại message ROS 2 không?
3. **ROS 2**: publisher và subscriber có nhìn thấy nhau, cùng domain và QoS tương thích không?
4. **RViz**: display có chọn đúng topic, đúng fixed frame và có TF hợp lệ không?

Cách chia lớp này giúp tránh nhầm lẫn giữa lỗi sensor, lỗi chuyển đổi message, lỗi middleware và lỗi hiển thị.

6 Kết luận

Luồng Gazebo-ROS 2-RViz gồm hai miền truyền thông nối bởi một bridge có kiểu. Gazebo dùng Protobuf trên Gazebo Transport; ROS 2 dùng message chuẩn, CDR và DDS; RViz chỉ làm việc với phía ROS 2. Bridge dịch nội dung của message chứ không chuyển tiếp nguyên byte. Trong đường live không có file raw bắt buộc. Khi cần lưu trữ, rosbag2/MCAP ghi lại message ROS 2, còn packet capture ghi lại dữ liệu ở lớp mạng. Hiểu rõ ranh giới này là nền tảng để thiết kế, quan sát và debug hệ thống robot mô phỏng.

Tài liệu tham khảo

- [1] Google, “Protocol Buffers Documentation.” <https://protobuf.dev/overview/>

- [2] Gazebo, “Gazebo Transport.” <https://gazebo.org/api/transport/13/>
- [3] Gazebo, “ROS 2 Integration: ros_gz_bridge.” https://github.com/gazebo/ros_gz/tree/jazzy/ros_gz_bridge
- [4] Open Robotics, “ROS 2 middleware implementations.” <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Different-Middleware-Vendors.html>
- [5] Open Robotics, “RViz User Guide.” <https://github.com/ros2/rviz/tree/rolling/rviz2>